

HELLO WORLD

THEORY / TEXT / IMAGE

THEORY	TEXT	IMAGE
What must be understood to explain, predict, and modify the program.	The executable p5.js representation: variables, functions, conditions, state changes.	What visual inspection can reveal - and what one frame necessarily hides.

A visual paper and working reference for p5.js program theory

Peter Naur x p5.js v2

"the proper, primary aim of programming is, not to produce programs, but to have the programmers build theories"

Peter Naur, "Programming as Theory Building" (1985), p. 253. [1]

REFERENCE PURPOSE

Use this document beside a sketch. Separate what you can see, what the text states, and what you must know to change the system correctly.

Examples target p5.js v2 APIs; source pages checked 2026-08-24. The p5.js download page recommends the 2.x line. [7]

THE INSPECTION METHOD

Code beside output is only a two-part explanation. Add the theory as a third inspectable object.

THEORY	TEXT	IMAGE
• causality • time • persistent state • alternatives • rationale • prediction • what must survive change	• setup() / draw() • variables • assignments • conditions • functions • events • library calls	• current frame • visible position • color / form • one realized branch • current result • perceptible behavior

CORE RELATION

Θ = theory T = program text I_t = visible image at time t E_t = execution through time

The test is not whether Θ , T, and I agree at one instant. The test is whether the learner can use their relation to predict and make an unforeseen modification.

1 LOOK	2 PREDICT	3 RUN	4 REVISE
-----------	--------------	----------	-------------

Naur's useful distinction is not "code versus mental model" in the abstract. His theory is the understanding that lets a programmer explain why the program has its form and respond constructively to new demands. [1]

HELLO WORLD: A STATIC IMAGE CAN HIDE AN ACTIVE LOOP

THEORY	PROGRAM TEXT	VISIBLE IMAGE
LIFECYCLE p5.js calls setup() once and then repeatedly calls draw(). HIDDEN AGENT The framework supplies the call schedule. The programmer does not write draw(); draw(); draw(); FORMAL same pixels at t1 and t2 does not mean no execution occurred between them.	<pre>function setup() { createCanvas(400, 400); } function draw() { background(220); }</pre>	VISIBLE FRAME: STATIC APPEARANCE / ACTIVE LOOP 
INSPECT Look only at the canvas. Can you determine whether it was painted once or thousands of times? Now inspect the text: where is draw() explicitly called? The missing relation is supplied by the p5.js runtime.	REFERENCE RULE VISIBLE STASIS != EXECUTION STASIS	

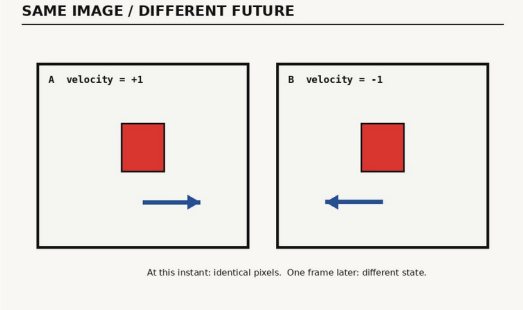
Sources: [2] p5.js setup() reference; [7] p5.js download/version guidance.

A RECTANGLE ACQUIRES A PAST: POSITION BECOMES STATE

THEORY	PROGRAM TEXT	VISIBLE IMAGE
IDENTITY THROUGH TIME x is not merely a coordinate. Its old value becomes input to its next value. STATE TRANSITION $x(t+1) = x(t) + 1$ WHAT THE FRAME HIDES direction, previous position, rate of change, and the fact that the same variable persists.	<pre>let x = 40; function setup() { createCanvas(400, 200); } function draw() { background(245); fill("red"); rect(x, 60, 80, 80); x = x + 1; }</pre>	VISIBLE FRAMES: ONE OBJECT, PERSISTENT STATE
INSPECT Freeze one frame. Can you infer how the rectangle arrived there? Predict x for the next five frames before running the sketch. Then change +1 to -1 and explain the consequence before execution.	REFERENCE RULE A FRAME SHOWS STATE; THEORY EXPLAINS TRANSITION	

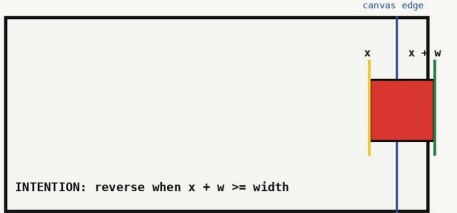
Sources: [3] Quiñones & Amarisa, "Variables and Change."

THE SAME IMAGE CAN HAVE DIFFERENT FUTURES

THEORY	PROGRAM TEXT	VISIBLE IMAGE
CURRENT IMAGE Both programs can display the rectangle at x = 150. LATENT DIFFERENCE velocity = +1 versus velocity = -1 COUNTERFACTUAL KNOWLEDGE Understanding is demonstrated by predicting what has not happened yet.	<pre>// Program A let x = 150; let velocity = 1; function draw() { background(245); rect(x, 60, 80, 80); x += velocity; } // Program B changes only: // let velocity = -1;</pre>	SAME IMAGE / DIFFERENT FUTURE 
INSPECT Do not run the programs. At the captured frame the pixels match. State which direction each rectangle will move next and identify the single value that determines the difference.	REFERENCE RULE APPEARANCE UNDERDETERMINES FUTURE BEHAVIOR	

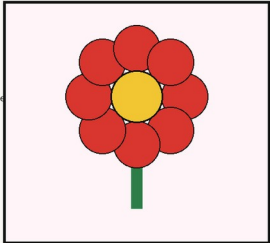
Sources: [3] Variables and Change; [1] Naur on explanation and modification.

DEBUGGING: THE BUG IS A DISAGREEMENT BETWEEN THEORY AND EXECUTION

THEORY	PROGRAM TEXT	VISIBLE IMAGE
INTENTION Reverse when the rectangle touches the right boundary. TEXTUAL TEST <code>x > width</code> tests the left edge after it has already crossed the canvas. REVISED THEORY <code>rightEdge = x + w</code>. The boundary event is <code>rightEdge >= width</code>.	<pre>let x = 40; let velocity = 2; let w = 80; function draw() { background(245); rect(x, 60, w, 80); x += velocity; // BUG if (x > width) { velocity *= -1; } }</pre> FIX / VARIANT <pre>// FIX if (x + w >= width) { velocity *= -1; }</pre>	BUG: x IS NOT THE RIGHT EDGE  INTENTION: reverse when <code>x + w >= width</code>
INSPECT Before patching, name what <code>x</code> denotes geometrically. Draw the right edge as <code>x + w</code>. A useful debugging repair changes both the code and the explanation of why the condition is correct.	REFERENCE RULE PATCH THE CAUSAL MODEL, NOT ONLY THE SYMPTOM	

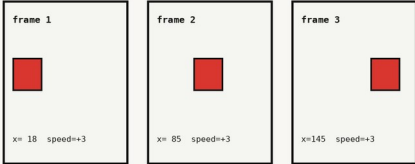
Sources: [4] p5.js Field Guide to Debugging.

EIGHT CIRCLES ARE NOT YET A FLOWER

THEORY	PROGRAM TEXT	VISIBLE IMAGE
STRUCTURE The drawing can be exhaustively described as geometric primitives. INTENTION The same primitives may be intended as petals, center, and stem. LIMIT Recovering geometry is not the same as recovering why that representation exists.	<pre>function setup() { createCanvas(300, 300); noLoop(); describe("A flower with red petals " + "and a green stem."); } function draw() { background(250); fill("green"); rect(145,145,10,110); fill("red"); // eight petal circles ... fill("yellow"); circle(150,145,55); }</pre>	GEOMETRY / INTENTION STRUCTURAL 8 red circles 1 yellow circle 1 green rectangle  INTENTIONAL FLOWER petals center stem
INSPECT Ask three separate questions: What primitives are drawn? What do they depict? Why is that depicted object here? The p5.js accessibility tutorial explicitly distinguishes automatic shape descriptions from authored descriptions of intended visual objects.	REFERENCE RULE STRUCTURAL COMPLETENESS != INTENTIONAL COMPLETENESS	

Sources: [6] Lichlyter & Quiñones, "Writing Accessible Canvas Descriptions."

GENERATED TEXT CAN EXIST BEFORE THE PROGRAMMER'S THEORY

THEORY	PROGRAM TEXT	VISIBLE IMAGE
NEW ORDER intention -> prompt -> model -> program text -> human reconstruction WHAT RUNNING PROVES Only that some execution path is valid enough to produce behavior. APPROPRIATION TEST Can the recipient explain the state and extend it under a demand not contained in the prompt?	<pre>let x = 0; let speed = 3; let w = 60; function setup() { createCanvas(400, 200); } function draw() { background(240); fill("red"); rect(x, 60, w, 60); x += speed; if (x <= 0 x + w >= width) { speed *= -1; } }</pre>	PROGRAM TEXT CAN ARRIVE BEFORE PROGRAM THEORY  Running code is evidence of execution; it is not yet evidence of understanding.
INSPECT After receiving generated code, remove the generator. Ask: why is speed signed? Why is w added to x? Then request: "On every third collision, pause for one second." The new demand forces invention of collision count and waiting state.	REFERENCE RULE EXECUTABLE TEXT IS NOT YET POSSESSED THEORY	

Sources: [5] Ciston, Martinez & Atairu, "Chatting with/about Code"; [1] Naur.

FROM MONDRIAN BLOCKS TO A THEORY INSPECTOR

THEORY	PROGRAM TEXT	VISIBLE IMAGE
DO NOT DISCARD DIRECT MANIPULATION A block can still map directly to rect(x,y,w,h). ADD INSPECTABLE TEMPORAL FACES STATE / LOOP / EVENT / MEMORY reveal what cannot be read from a static rectangle. DESIGN GOAL Move among image, executable text, and causal theory without pretending any one view contains the others.	<pre>let A = { x: 40, y: 60, w: 120, h: 140, velocity: 2, collisionCount: 0 }; function draw() { background(248); A.x += A.velocity; if (A.x + A.w >= width) { A.velocity *= -1; A.collisionCount++; } rect(A.x, A.y, A.w, A.h); }</pre>	THEORY INSPECTOR A / THEORY STATE x = 40 velocity = 2 LOOP each frame: x <- x + velocity EVENT when rightEdge >= boundary THEN velocity <- -velocity MEMORY previousX / collisionCount
INSPECT Drag the rectangle to change geometry. Then open STATE to see persistence, LOOP to see recurrence, EVENT to see branch conditions, and MEMORY to see history-dependent behavior. A new requirement should force the learner to add or revise one of these relations.	REFERENCE RULE MAKE THE RELATION INSPECTABLE	

Sources: [1] Naur; [2]-[6] p5.js tutorials and references used as operational cases.

REFERENCE: USE THIS BESIDE A SKETCH

A compact inspection sheet for teaching, code review, or debugging.

1 LIFECYCLE Who calls this? Once, every frame, or on an event? <code>setup()</code> once <code>draw()</code> repeatedly <code>mousePressed()</code> on event	2 STATE What survives from one execution to the next? <code>x</code> , <code>velocity</code> , <code>count</code> , <code>mode</code> , <code>history</code>	3 TRANSITION What equation or rule produces the next state? <code>x_next = x + velocity</code>
4 CONDITION Which alternate future becomes active, and why? <code>if (rightEdge >= width) reverse</code>	5 REPRESENTATION What does each variable or object stand for? <code>x</code> = left edge, not “the whole rectangle”	6 RATIONALE What must remain true after modification? the rectangle stays inside the canvas

THE MODIFICATION TEST

Do not finish with “reproduce the sketch.” Ask for a behavior the representation has not already demonstrated.

PREDICT What must persist?	EXPLAIN Why can't the current system satisfy the request?	MODIFY What new state / rule is required?	TRANSFER Can the reasoning survive a new sketch?
--------------------------------------	---	---	--

PROMPTS THAT EXPOSE THEORY

- Make the rectangle remember where it was five seconds ago.
- Reverse only on every third collision.
- Make the animation behave consistently when frame rate changes.
- Make two rectangles obey the same rule without duplicating the behavior.
- Explain why the output is correct even though the image appears unchanged.

REFERENCE: FIVE p5.js PATTERNS

Minimal patterns to keep the execution theory explicit.

1. LIFECYCLE setup() initializes; draw() is framework-scheduled repeated execution. [2]	<pre>function setup() { createCanvas(400, 200); } function draw() { // repeated work }</pre>
2. PERSISTENT STATE Declare outside draw() when a value must survive frames. [3]	<pre>let x = 40; function draw() { x += 1; }</pre>
3. BOUNDARY MODEL Name the geometry you actually test.	<pre>let rightEdge = x + w; if (rightEdge >= width) { velocity *= -1; }</pre>
4. DEBUG TRACE Inspect values and flow instead of guessing. [4]	<pre>console.log({ frame: frameCount, x, velocity });</pre>
5. ACCESSIBLE INTENTION When useful, author a concise description of the visual result. [6]	<pre>describe("A flower with red petals " + "and a green stem.");</pre>

REFERENCES & SOURCE NOTES

[1] Naur, Peter. 1985. "Programming as Theory Building." *Microprocessing and Microprogramming* 15(5): 253-261. DOI: 10.1016/0165-6074(85)90032-8. Primary claim used here: programming has a primary aim of building theories of how problems are handled by program execution.

<https://www.naur.com/comp/c1-4.html>

[2] p5.js Contributors. "setup()" reference. Explains that setup() is called once automatically and draw() begins looping afterward.

<https://p5js.org/reference/p5/setup/>

[3] Quiñones, Layla, and Joanne Amarisa. "Variables and Change." p5.js Tutorials. Used for variables, scope, repeated draw(), and animation through changing coordinates.

<https://p5js.org/tutorials/variables-and-change/>

[4] McIntyre, Nick, et al. "Field Guide to Debugging." p5.js Tutorials. Used for the mismatch between the programmer's mental picture and observed execution, and for inspecting variables/program flow.

<https://p5js.org/tutorials/field-guide-to-debugging/>

[5] Ciston, Sarah, Emily Martinez, and Minne Atairu. "Chatting with/about Code." p5.js Tutorials. Used for AI-generated p5.js code, pseudocode, inspection, adaptation, and the warning not to assume generated code works.

<https://p5js.org/tutorials/criticalAI1-chatting-with-about-code/>

[6] Lichlyter, Kathryn, and Layla Quiñones. "Writing Accessible Canvas Descriptions." p5.js Tutorials. Used for the distinction between automatic structural output and authored descriptions that can express visual intention.

<https://p5js.org/tutorials/writing-accessible-canvas-descriptions/>

[7] p5.js Contributors. "Download." Current p5.js 2.x CDN/version guidance. Examples in this reference use APIs compatible with p5.js v2.

<https://p5js.org/download/>

HOW TO CITE THIS WORKING REFERENCE

"Hello World: Theory / Text / Image. A Visual Paper and Working Reference for p5.js Program Theory." Working paper, 2026.

FIGURE STATUS

All figures in this PDF are original reference renders created for the code examples in this document. They depict the specified canvas states; they are not screenshots copied from p5.js tutorial pages.

VERSION NOTE

Source pages were checked on 2026-08-24. p5.js evolves; verify API details against the current p5.js reference when using this document as a coding reference.